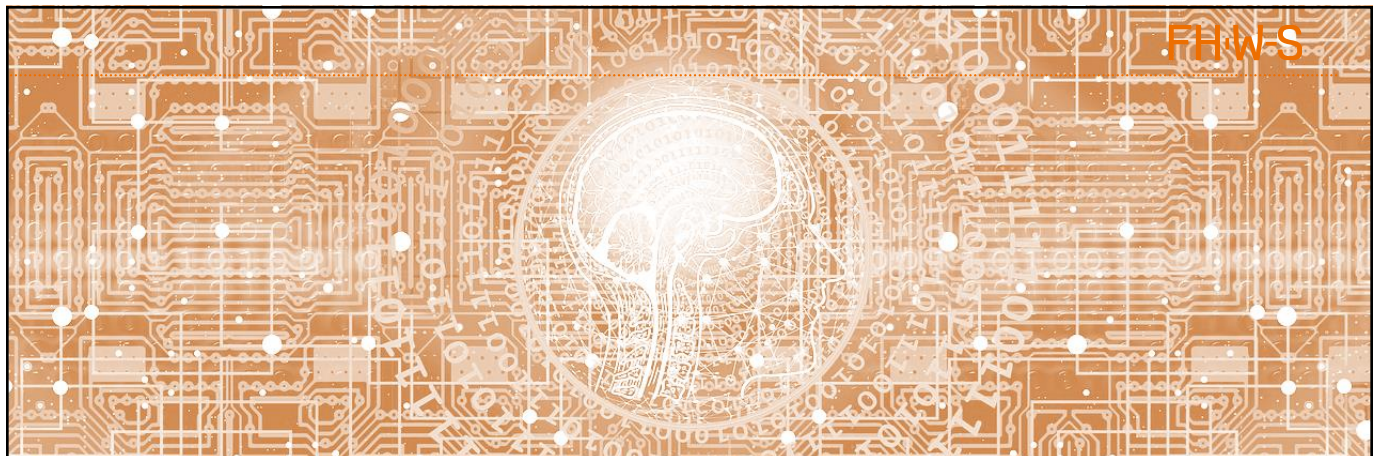




Software Engineering and Cyber Security

Prof. Dr. Marian Daun

FHWS University of Applied Sciences
Würzburg-Schweinfurt

The slide has a background image of a desk with a wooden surface. On the desk is an open notebook with a calendar page showing the number '14'. A black pen and a silver smartphone are also on the desk. A semi-transparent white box with an orange border contains the agenda items. The FHWS logo is in the bottom left, and the number '2' is in the bottom right.

Agenda

- Software Engineering Fundamentals
- Requirements Engineering
- Architecture Design
- Modelling
- Development and Implementation
- Software Quality Assurance

FHWS University of Applied Sciences
Würzburg-Schweinfurt

2

Architecture Design

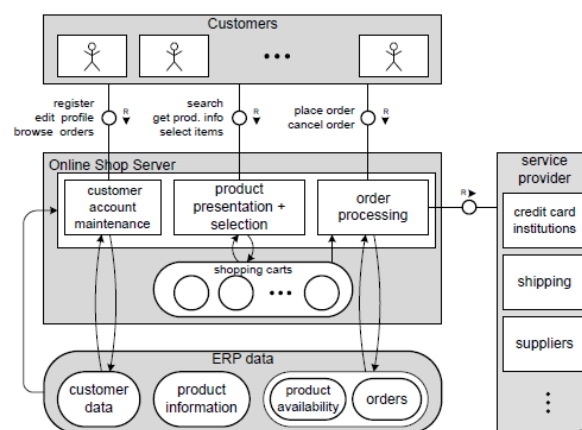


Group Discussion

Why do we need Software Architecture?

Terminology

What is Software Architecture?



What is Software Architecture?

“People in the software world have long argued about a definition of architecture. For some it's something like the fundamental organization of a system, or the way the highest level components are wired together. My thinking on this was shaped by an email exchange with Ralph Johnson, who questioned this phrasing, arguing that there was no objective way to define what was fundamental, or high level and that a better view of architecture was **the shared understanding that the expert developers have of the system design**.

A second common style of definition for architecture is that it's “the design decisions that need to be made early in a project”, but Ralph complained about this too, saying that it was more like **the decisions you wish you could get right early in a project**.

His conclusion was that **“Architecture is about the important stuff. Whatever that is”**. On first blush, that sounds trite, but I find it carries a lot of richness. It means that the heart of thinking architecturally about software is to decide what is important, (i.e. what is architectural), and then expend energy on keeping those architectural elements in good condition. For a developer to become an architect, they need to be able to recognize what elements are important, recognizing what elements are likely to result in serious problems should they not be controlled.

” [Martin Fowler]

What is Software Architecture?



Group Discussion





Group Discussion

Where is the difference to Requirements Engineering?

Be careful, people easily get offended by this question

Rationals for Architecture Decisions

Obviously: Requirements Specification

Architecture Decisions are mostly driven by

- Non-functional properties
- Planned product life cycle
- Re-use of software
- Number of users
- Compatibility

Components

Modular Architecture

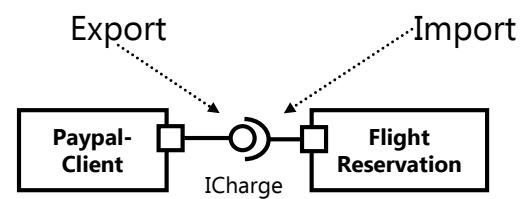
Components and relationships are the major elements of a software architecture

Component

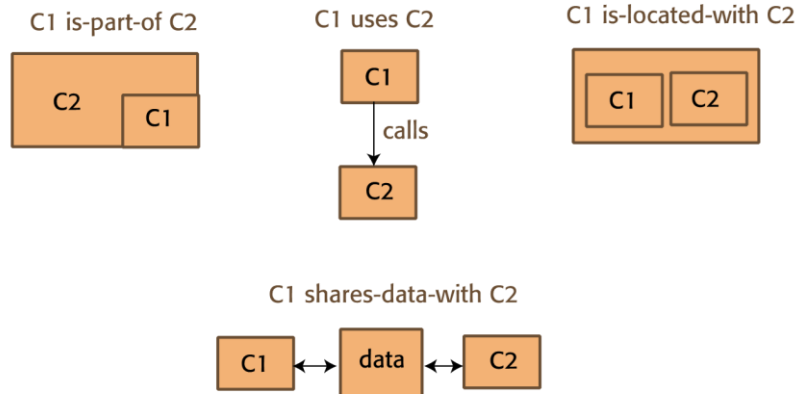
Self-contained software unit
Defines functions and services

Relationships

Services provided
Services consumed



Component Relations



[Sommerville 2018]

Defining a Modular Architecture

Top-down-Design

Often used when no or little knowledge about the basic components exists

Greenfield Approach

Components are decomposed until manageable and realisable set of fine-grained components is reached

Bottom-up-Design

Used when many components shall be re-used or the technical layer is already well known

Brownfield Approach

Components are composed until a further composition does not result in an easier description of the system



Exercise

Define and Structure Components of a Webshop

Guidelines

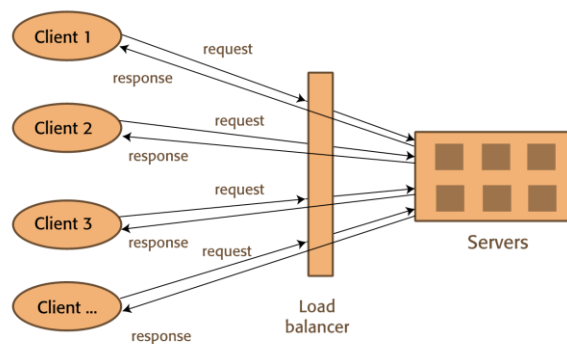
Separation of concerns

Stable interfaces

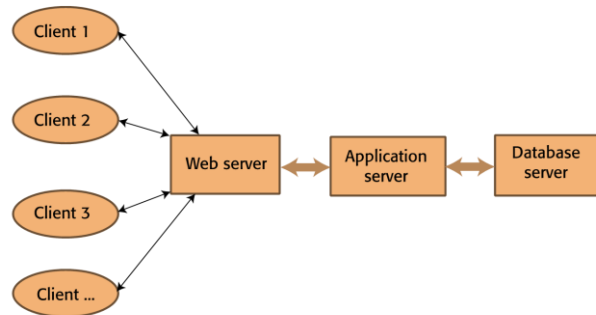
No redundancy (implement only once)

Well-known Architectures

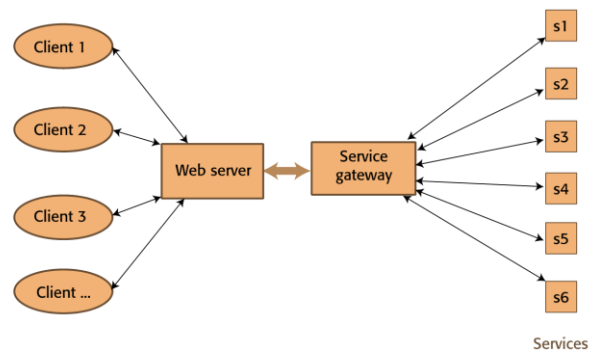
Client-Server Architecture



Multi-tier Client-Server Architecture



Service-oriented Architecture





Exercise

For next week prepare a presentation for:

Agile Architecture (<https://martinfowler.com/agile.html>)

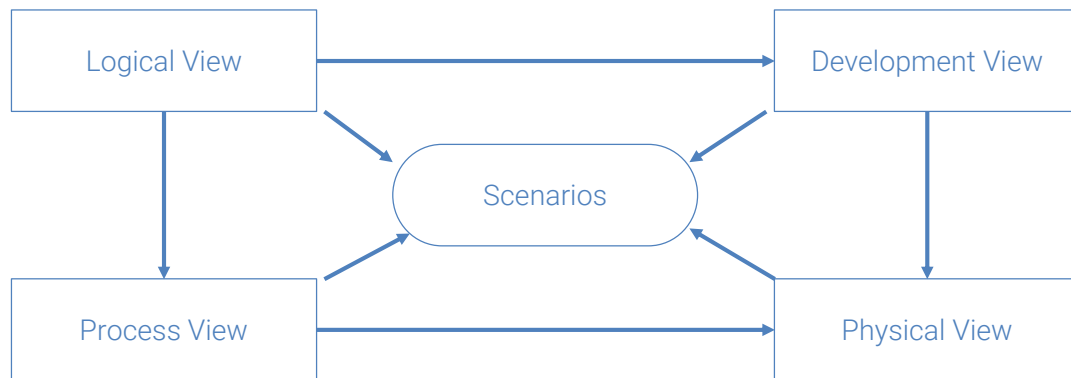
Microservice Architecture

(<https://martinfowler.com/articles/microservices.html>)

DevOps (<https://www.dynatrace.com/news/blog/what-is-devops/>)

Views

Kruchten's 4+1 Model



Group Discussion

Why do we need views?

ISO 42010

“Read”: ISO/IEC/IEEE 42010 *Systems and Software Engineering – Architecture Description*

Difference between View and Viewpoint

Architecture Patterns

Architecture Patterns

Re-use of architectures

Established, well-working structures are documented as architecture patterns

Architecture patterns are re-used to create good architectures with low effort

Multi-tier architecture (Schichtenarchitektur)

Tiers build upon one another.
Access crossing multiple tiers is forbidden.
Only neighboring tiers can access each other.

Changes should commonly affect only one tier.

Common Layers

Presentation Layer

Application Layer

Business Layer

Data Access Layer

Three-tier Architecture

Presentation Tier

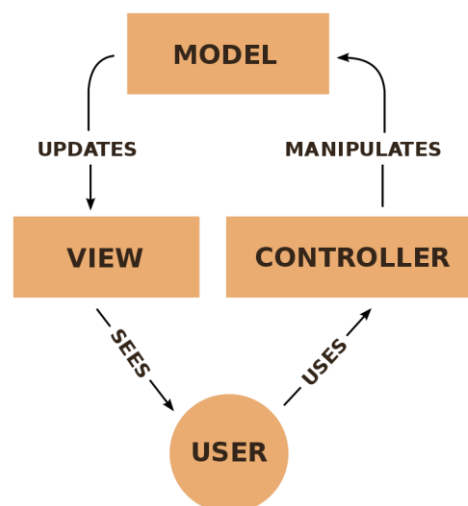
Logic Tier

Data Tier

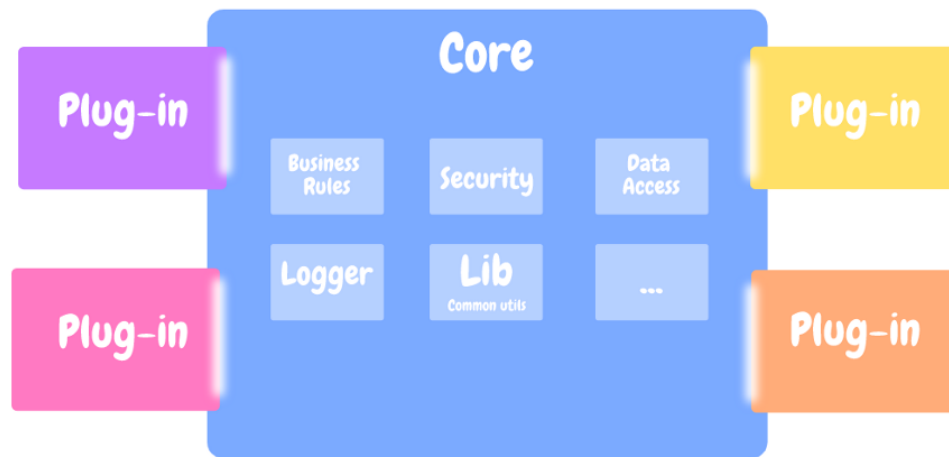
Pipe-and-Filter



Model View Controller



Plug-in Architecture



<https://medium.com/omarelgabrys-blog/plugin-architecture-dec207291800>

Questions for Self-Assessment

- What is an architecture?
- What is the difference between requirements engineering and architecture design? Where are overlaps?
- When are top-down and bottom-up design approaches used?
- What influences an architecture?
- What is a component-based architecture?
- What is a view?
- What are important architecture patterns and when should they be used?